Early Wildfire Detection in Southern California Using Computer Vision: Convolutional Neural Networks with TensorFlow and Keras to Save Lives and Preserve Communities

Holly Do

Figure 1: Southern California Wildfire 2025

Part 1: Defining the Problem

Southern California wildfires have cost the state of California hundreds of billions of dollars in damages and destroyed the homes of tens of thousands of civilians over the past five years. Because wildfires often display unpredictable patterns influenced by unforeseen factors such as Santa Ana winds, global warming, vegetation changes, and human activities, they are very difficult to forecast accurately. To address this problem, the Los Angeles County Office aims to develop a solution that focuses on identifying fires at their earliest stages, especially those caused by human behavior, to enable faster containment and minimize destruction. This machine learning project therefore focuses on early fire detection using drone-based computer vision for constant surveillance during the wildfire riskiest months.

Most existing disaster management relies on historical data and manual calculations that average past occurrences to allocate preventive measures. This method assumes each year is like the last; however, it fails to account for dynamic and unpredictable factors such as changing wind conditions, global warming, vegetation growth, and expanding human development. Since human activity accounts for most wildfire ignitions, detecting early signs of fire can significantly reduce the

damage caused by wind-driven spread and dry Californian weather that threaten human lives and destroy properties.

In developing this model, several assumptions and guidelines are established. It is assumed that most Southern California wildfires originate from human activity or related accidents. It also assumed that the dataset sufficiently captures nuanced urban activities in Southern California. A supervised learning approach will be used, with labeled data to classify smoke and fire ignition scenes from everyday urban environments. The training and validation phases will involve standard hyperparameter tuning for image-based convolutional neural networks, including adjustments to the learning rate, number of neurons, dropout rate, batch size, and activation functions. Model performance will be evaluated using standard supervised learning metrics such as accuracy and F1 score. Disaster experts may also contribute by refining the dataset and providing more nuanced quality image data.

The project will require a dataset resembling images captured by surveillance drones. This dataset should contain approximately 600 to 10,000 public images depicting both fire and disaster scenes as well as typical urban settings with vehicles and people. The dataset will be labeled and divided into training, validation, and test sets for model development and hyperparameter tuning. Once a potential wildfire or smoke ignition is detected, an automated alert will be sent to the nearest fire station, allowing authorities to inspect the scene in real time and deploy firefighting personnel and resources effectively and minimize further destruction.

In conclusion, this CNN-based drone surveillance project aims to proactively identify wildfire risks and detect ignitions at the earliest possible stage. Early detection is essential for minimizing loss of life and property damage caused by the increasingly uncontainable wildfire conditions that ravage Southern California each year. Through the thoughtful application of technology and machine learning, this project demonstrates how innovation can serve a meaningful purpose—protecting communities, saving lives, and ultimately improving human condition.

Part 2: Data Analysis Report

1. Data Definition:

The dataset that i found consists of RGB images of urban disaster scenarios, including fire, vehicles, and humans. The images are originally 640x640 pixels with 24-bit depth. For each channel, pixel intensity values are normalized to the range [0,1] for each of the three color channels—red, green, and blue—by dividing by 255.

The target labels are integers from 0 to 6, representing six types of categories. These labels are one-hot encoded for training, converting each integer label into a six-dimensional vector suitable for output. The total number of classes is six: "person", "fire", "smoke", "small_vehicle", "large_vehicle", "two_wheeler".

Key hyperparameters for training include the learning rate, which controls the step size in gradient descent (or use Adam optimizer), batch size, and the number of epochs, determining how many passes the model makes through the dataset.

The network architecture may include convolution and pooling layers and fully connected dense layers, with an optional dropout rate of 25% to prevent overfitting.

2. Plan for Missing Data

For missing images, additional data should be collected, especially for underrepresented disaster categories.

For missing labels, these should be manually verified and filled to ensure accurate training.

3. Plan for Additional Parameters or Data

The current dataset represents typical urban settings near Los Angeles forestry areas. If other disaster types are relevant or frequent, additional categories should be collected to enable the model to recognize these categories.

4. Necessary Transformations

Normalization is applied to scale pixel values to [0,1] for faster convergence. Each 640x640 image into a small 64x64 format, matched in order with the labels provided.

Optional transformations for advanced models include dimensionality reduction and standardization, though the latter may be unnecessary since pixel values are already normalized as stated above.

5. Plan for Separating Data

The dataset is split into a training set of 10,450 images, a validation set of 1,556 images, and a test set of 434 images. This ensures the model is trained, tuned, and evaluated on separate untouched subsets.

6. Visualization of Dataset Size

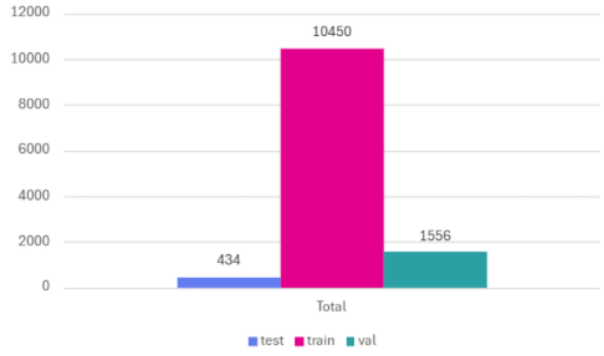

Figure 2: Sizes of train, validation, and test set. Original pictures are to be reduced from 640x640 to 64x64.

7. Plan for Hyperparameter Tuning

For optimizing model performance, several hyperparameters should be tested:

The number of neurons and layers can be adjusted and may improve accuracy but can lead to overfitting.

Dropout rates of 0.25 help prevent overfitting and better generalizarion.

Batch sizes of 32, 64, 128 improve learning speed and prevent the gradient to be biased toward one example.

Activation functions are options such as Relu, Sigmoid, or SoftMax.

Validation accuracy should guide the selection of optimal hyperparameters.

8. Training and Test Metrics

Model performance should be visualized using several key metrics:

Accuracy and loss statistics over epochs help identify underfitting or overfitting.

A confusion matrix provides insights into true positives, true negatives, false positives, and false negatives for each class.

The classification report offers detailed precision, recall, and F1-score values. Precision is number of correct positive predictions out of all positive predictions, recall is the proportion of correct positive predictions out of all actual positives, and the F1-score is the harmonic mean of precision and recall, useful for imbalanced datasets.

For multiclass classification, ROC curves and AUC values can visualize the trade-off between true positive rate and false positive rate.

Part 3: Three Models and their Hyperparameters

Model: 2 Conv Layers CNN

Two-layer CNN with architecture:

- Input: 64×64×3
- Two convolutional layers (32, 64 filters) with relu activation
- Dropout (25%) after convolution 2
- Fully connected layer: 128 units

- Output: 6 classes

Parameter	Value	Justification
Batch Size	10	Smaller batch size is used to moderate speed
Dropout after conv 2	0.25	Avoid eliminating too many connections
FC Dropout	0.25	Preserves more neurons in a small model
Epochs	10	Need to learn basic features for comparison

- Test Accuracy: ~81.8%
- Val Accuracy: ~0.50

Model converged quickly but is simple with low test accuracy score.

Model: 3 Conv Layers CNN

Three-layer CNN with architecture:

- Added Conv3 (128 filters) to previous
- FC layer: 128 units
- Dropout after Conv3 and FC layer

Parameter	Value	Justification
Batch Size	10	Smaller batch size is used to moderate speed
Dropout	0.25	Avoid eliminating too many connections
Dropout (FC)	0.25	Preserves more neurons in a small model
Epochs	10	Need to learn basic features for comparison

- Test Accuracy: ~78%
- Val Accuracy: ~0.50

Model converged quickly but is simple with low test accuracy score.

Model: 5 Conv Layers CNN + 2 Max Pooling

Five-layer CNN with architecture:

- Progressive filters: 32, 64, 64, 128, 256
- Two pooling layers (after Conv2 & Conv4) with filter size 2 and stride 2
- FC layer: 128 units

- Output: 6 classes

Parameter	Value	Justification
Batch Size	10	Smaller batch size is used to moderate speed
Dropout at FC	0.25	Preserves more neurons in a small model
Epochs	10	Need to learn basic features for comparison

- Test Accuracy: ~86%
- Val Accuracy: ~0.50

Model: 5 Conv Layers CNN + 2 Max Pooling + Dropout

Five-layer CNN with architecture:

- Progressive filters: 32, 64, 64, 128, 256
- Two pooling layers (after Conv2 & Conv4) with size 2 and stride 2
- Dropout at both pooling stages (25%)
- FC layer: 128 units
- Output: 6 classes

Parameter	Value	Justification
Batch Size	10	Smaller batch size is used to moderate speed
Dropout	0.25	Preserves more neurons in a small model
Dropout (FC)	0.25	Avoid eliminating too many connections
Epochs	10	Need to learn basic features for comparison

- Test Accuracy: ~87%
- Val Accuracy: ~0.50

This model achieved the highest accuracy among the four tested models thus we choose to further tune hyperparameters and increase the epoch number to achieve accuracy of above 90% when generalized on test set.

Part 4: Further Tune the Hyperparameters

Model: 5 Layer CNN + 2 Max Pooling + Dropout

Five-layer CNN with architecture:

- 5 layers of convolutional filters: 32, 64, 64, 128, 256
- Two pooling layers (after Conv2 & Conv4) with filter size 2 and stride 2
- Dropout at both pooling stages (25%)
- FC layer: 128 units
- Output: 6 classes

Parameter	Old Values	New Values Justification
Batch Size	10	Keep at 10 to moderate speed
Dropout	0.25	Improved generalization by adding dropout layers
Dropout (FC)	0.25	Increased to 0.5 to improve generalization
Epochs	20	Increased time training to improve accuracy
Test Accuracy	85%	92%
Val Accuracy	~0.5	~0.5

```
INFO:tensorflow:Restoring parameters from ./my_fire_model_rgb
✅ Final Test Accuracy: 0.9194
```

Conclusions and suggestions:

- Test Accuracy improves by a good margin
- Better generalization from adding dropout layers
- More epochs give more time to converge
- Model can be combined with others (ensembled model) for better results
- Combining with other existing popular architectures like YOLO, ResNet, or EfficientNet for better results

Part 5: Final CNN Machine Learning Model

Objective: Develop a convolutional neural network to decipher and classify camera images into categories like person, fire, smoke… to aid the city of Los Angeles with fire detection and risk management.

Achievement: The CNN ML model achieved **92% test accuracy** after hyperparameters tuning.

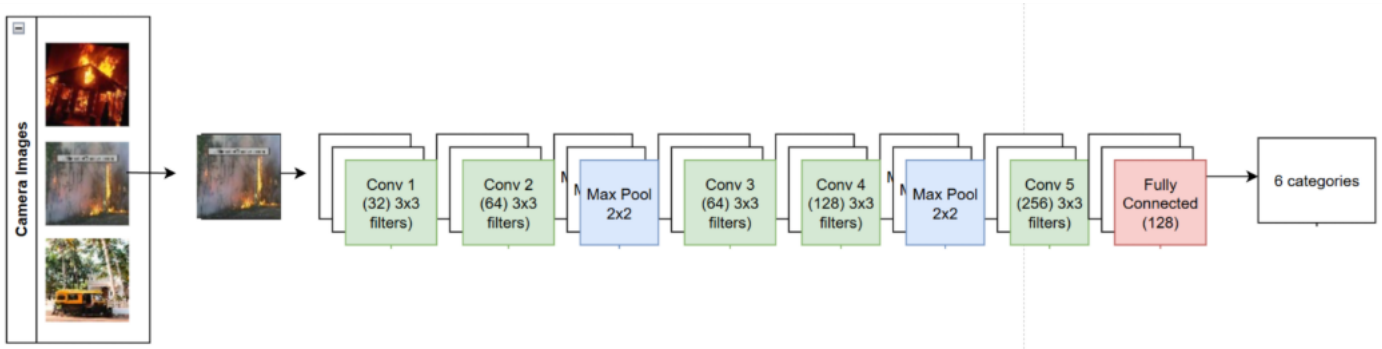

Figure 1: Overall flowchart of the convolutional neural network used in this presentation

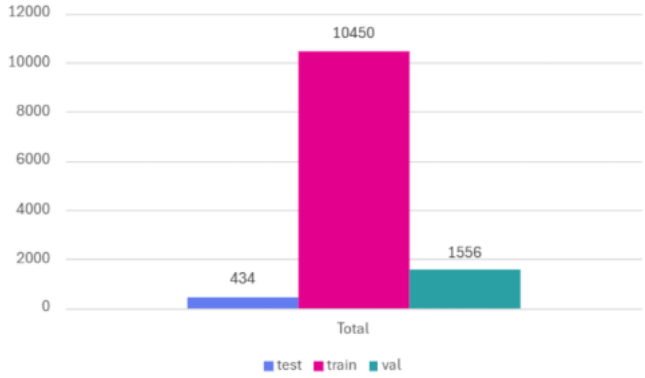

Figure 2: Sizes of train, validation, and test set. Original pictures are to be reduced from 640x640 to 64x64.

Future Applications and Improvements:

- Computer vision, face recognition, autonomous vehicles, and safety robotics.
- Opportunity to explore ensemble methods to further improve accuracy.
- Real-time optimization for deployment on low-power edge devices.
- Optimize model for real-time inference in practical applications combined with RNN, LSTM or GRU to capture frame-to-frame fire progression for integration into real-world systems.

Citation Source Dataset: Dataset from Rupankar Majumdar (Owner) on Kaggle.com

https://www.kaggle.com/datasets/rupankarmajumdar/disaster-response-object-detection-dataset